# How to Program your first FPGA Device

This tutorial shows you how to create the hardware equivalent of "Hello World": a blinking LED. This is a simple exercise to get you started using the Intel® Quartus® software for FPGA development.

You'll learn to compile VHDL code, make pin assignments, create timing constraints, and then program the FPGA to blink user LEDs on the board. You'll use a 50 MHz clock input (from the on-board oscillator) to drive a counter, and assign an LED to one of the counter output bits.

## Materials

### Hardware

Terasic DE10-Lite kit or DE1-SoC kit

The Terasic DE10-Lite development board, based on an Intel® MAX10 FPGA. You can buy the kit here.

The Terasic DE1-SOC development board, based on an Intel® Cyclone V FPGA. You can buy the kit here.

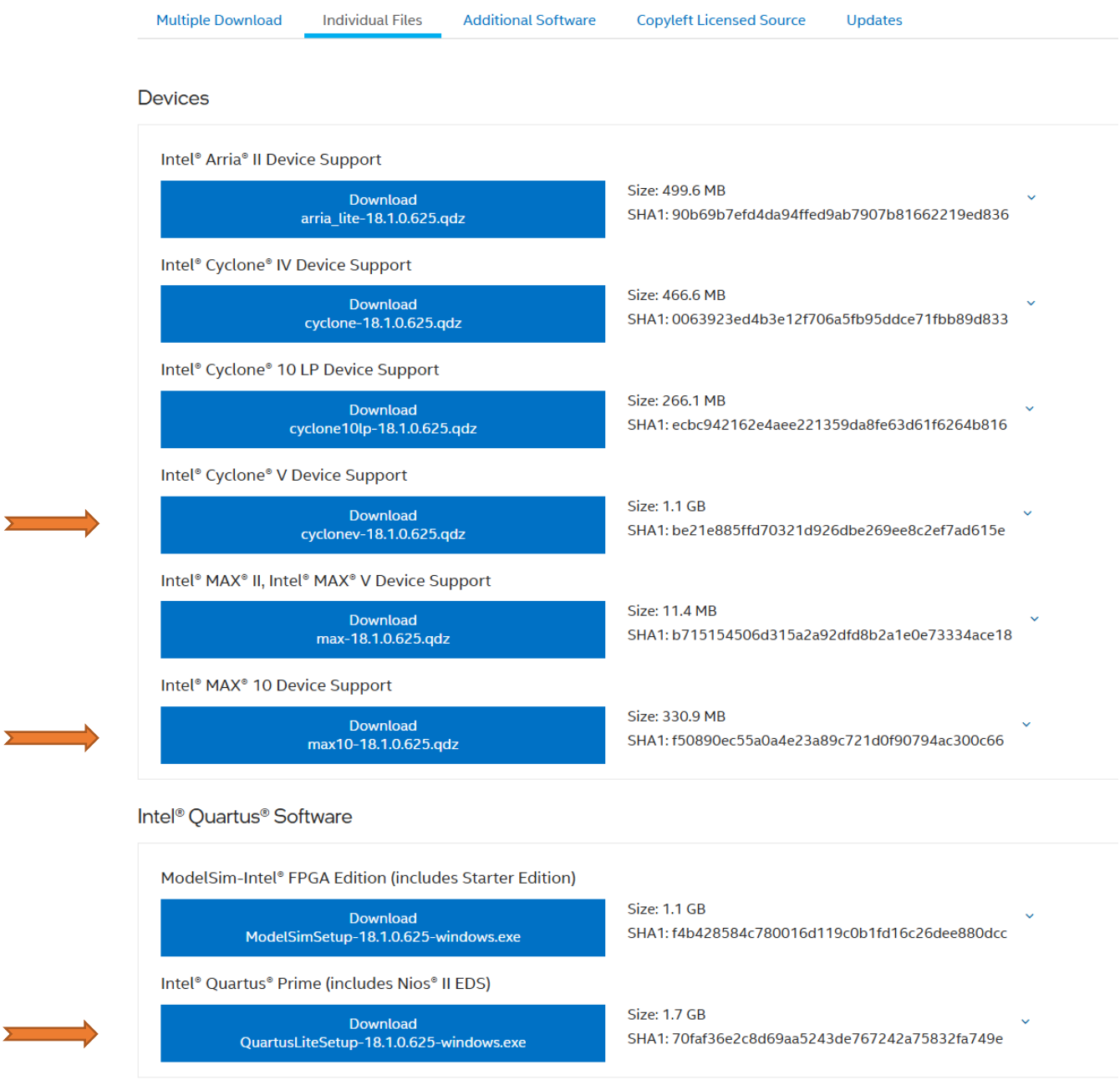
### Software

Intel® Quartus® Prime Software Suite Lite Edition

The FPGA design software used here is ideal for beginners as it's free to download and no license file is required. You can download the software here.

For this Tutorial, we are using Quartus Prime Lite version 18.1 for Windows.

> **Note**: The installation files are large (several gigabytes) and can take a long time to download and install. To minimize download time and disk space required, we recommend you download only those items necessary for this exercise. When prompted which files to download, uncheck "**Select All**" and select only **Quartus Prime Lite** and **Cyclone V and MAX10 device support** only.




Once you've downloaded and installed the Intel® Quartus® software, you're ready to get started creating a project!

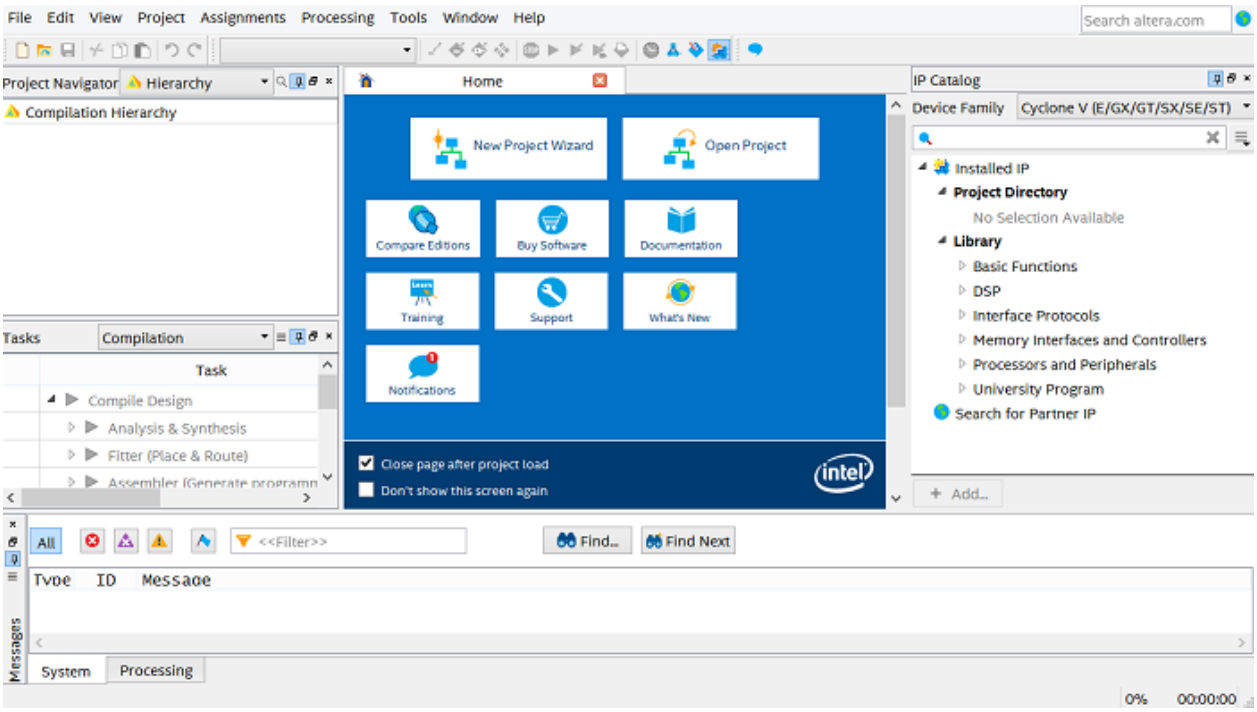
Why is the Quartus download so big?

The Quartus download contains several sophisticated tools to create a custom chip design, such as simulators, synthesis tools, place and route engines, timing analyzers, and device programmers, to name a few. Nearly all those functions are built into the Quartus Prime FPGA design software itself. The download also includes the embedded software design suite for the Nios II soft CPU, and one or more FPGA family databases - in our case the Cyclone V and MAX10 FPGA database.

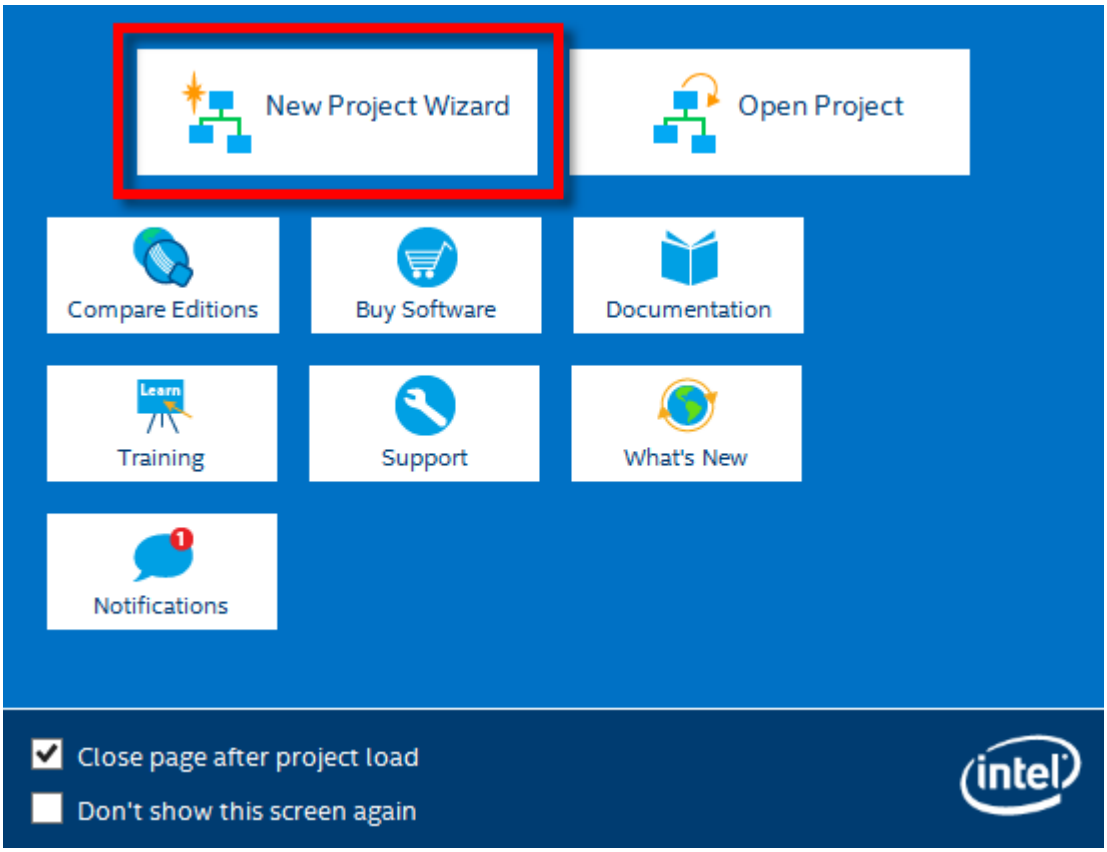
> **Note**: Screenshots are based on the latest release v18.1. User experience may vary when using earlier or later versions of Intel® Quartus® software.

# Step 1: Create an Intel® Quartus® Software Project

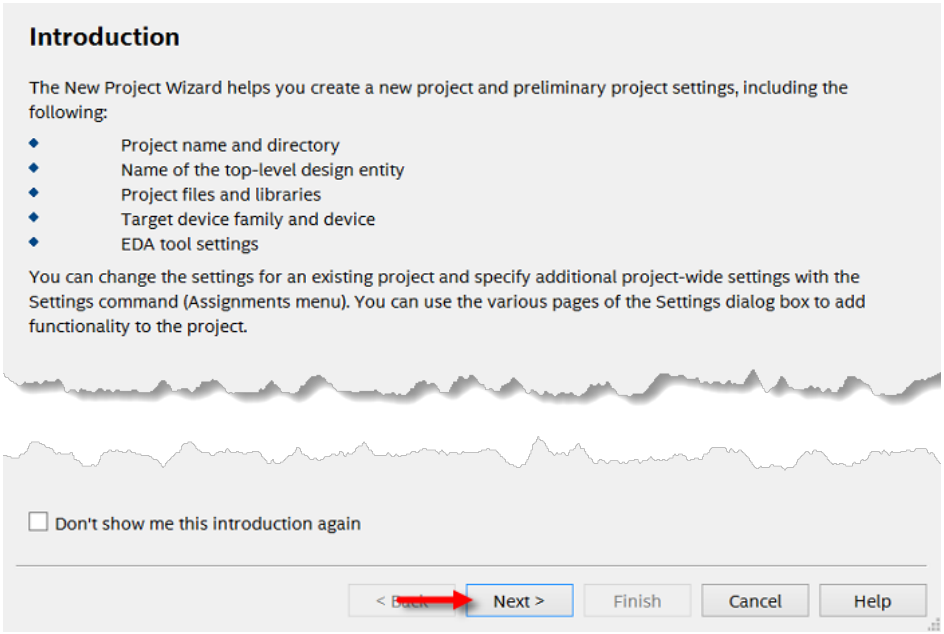
## Step 1.a: Open Intel® Quartus® Prime Software Suite Lite Edition.




## Step 1.b: Open a **New Project Wizard**

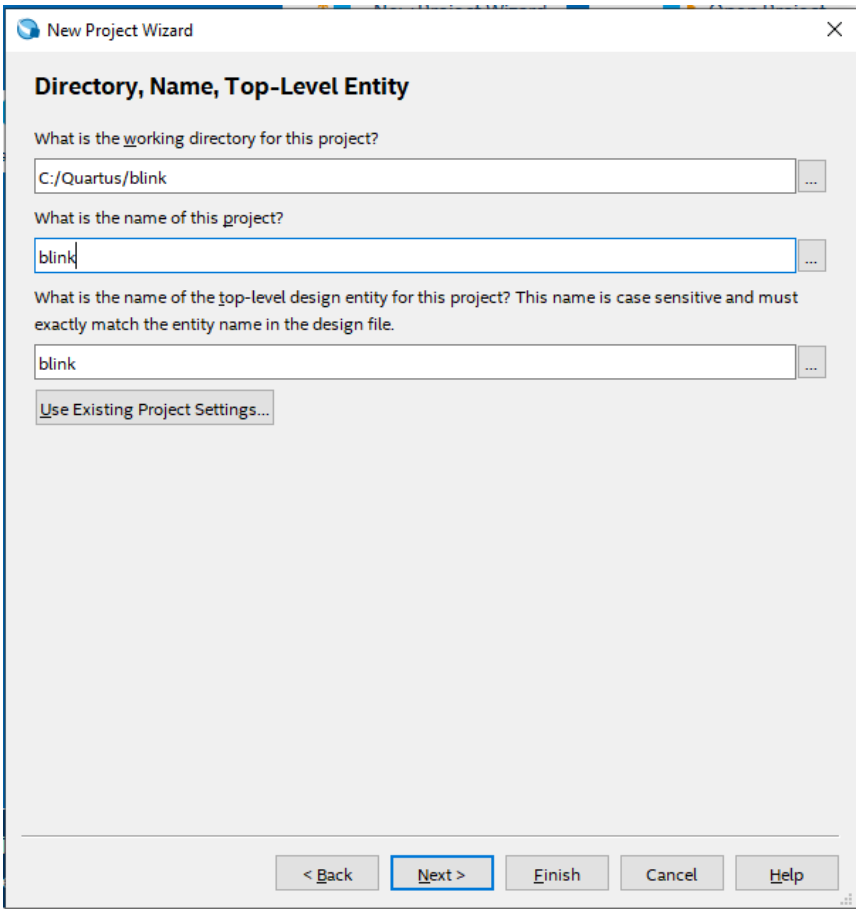



## Step 1.c: Select **Next**




## Step 1.d: Directory, Name, Top-Level Entity

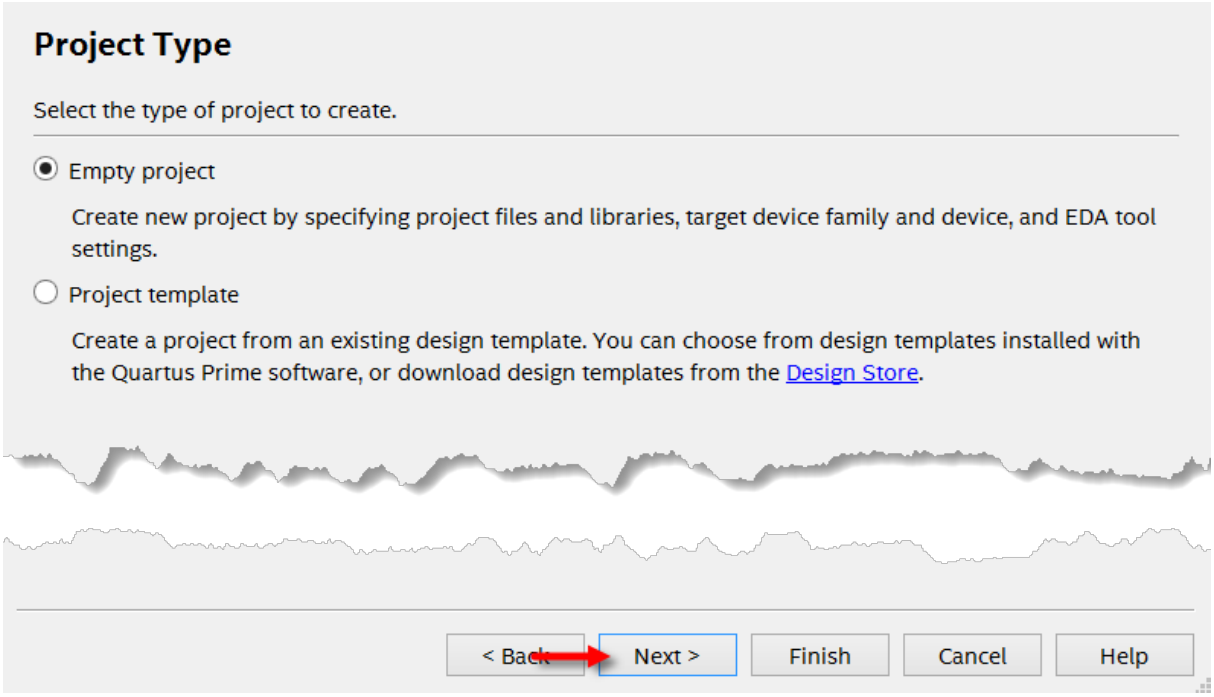
Choose a directory to put your project under. Here, we name our project "Blink" and place it under the C:/Quartus/blink folder but you can place it wherever you want. Select **Next**.




When prompted to create the directory, choose **Yes**.

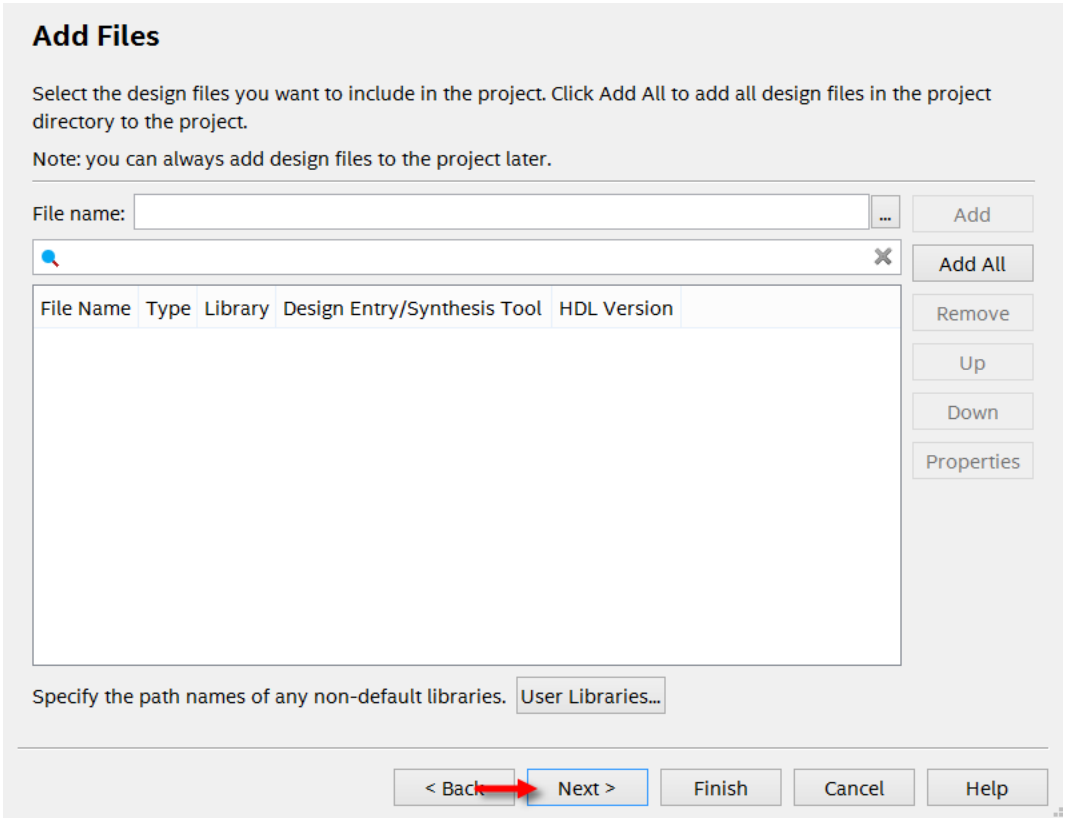
### Step 1.e: Project Type

Select **Empty Project,** and then click **Next**.




Step 1.f: Add Files

You won't be adding any files here. Click **Next**.



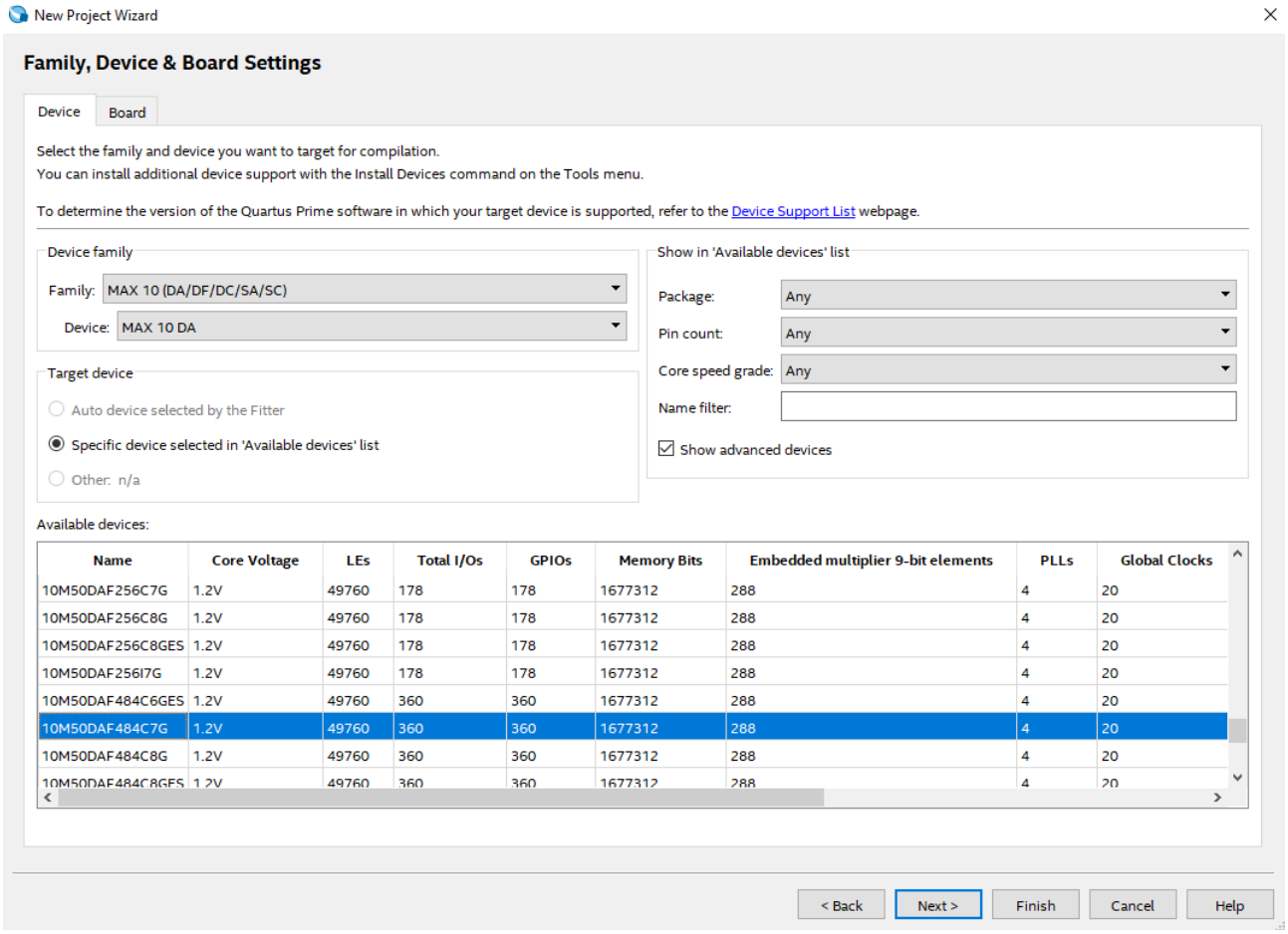

Step 1.g: Family, Device, and Board Settings (**Note:** You may need to expand window to view more device names)

For the DE10-Lite board, select the following:

*Family*: **MAX10**

*Device*: **MAX 10 DA**

*Device name*: **10M50DAF484C7G**




For the DE1-SoC board, Select the following:

*Family*: **Cyclone V**

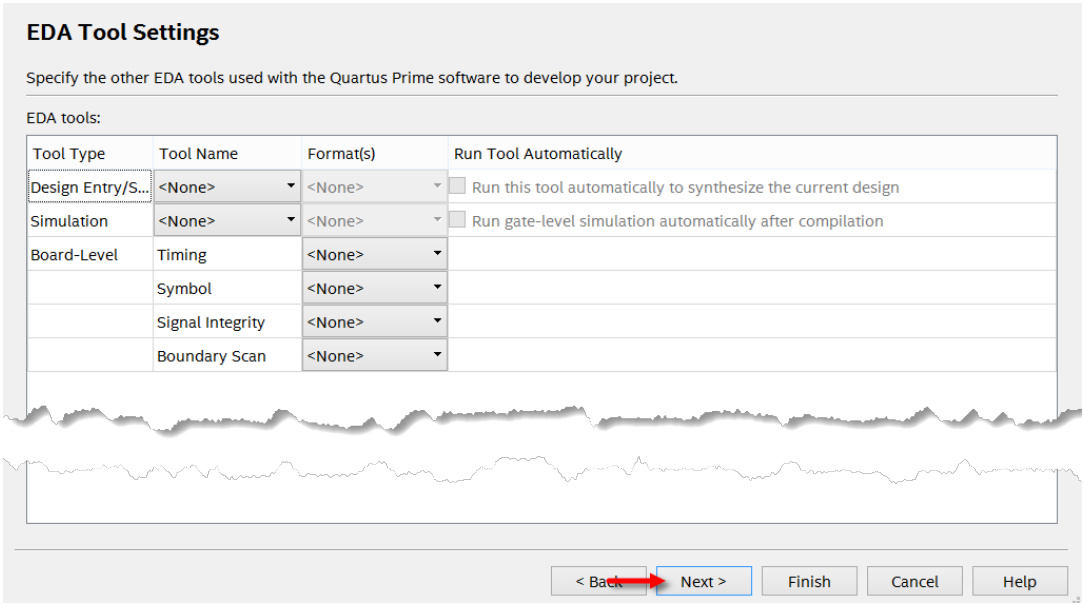
*Device*: **Cyclone V SE Mainstream**

*Device name*: **5CSEMA5F31C6**

Click **Next**.

### Step 1.h: EDA Tool Settings

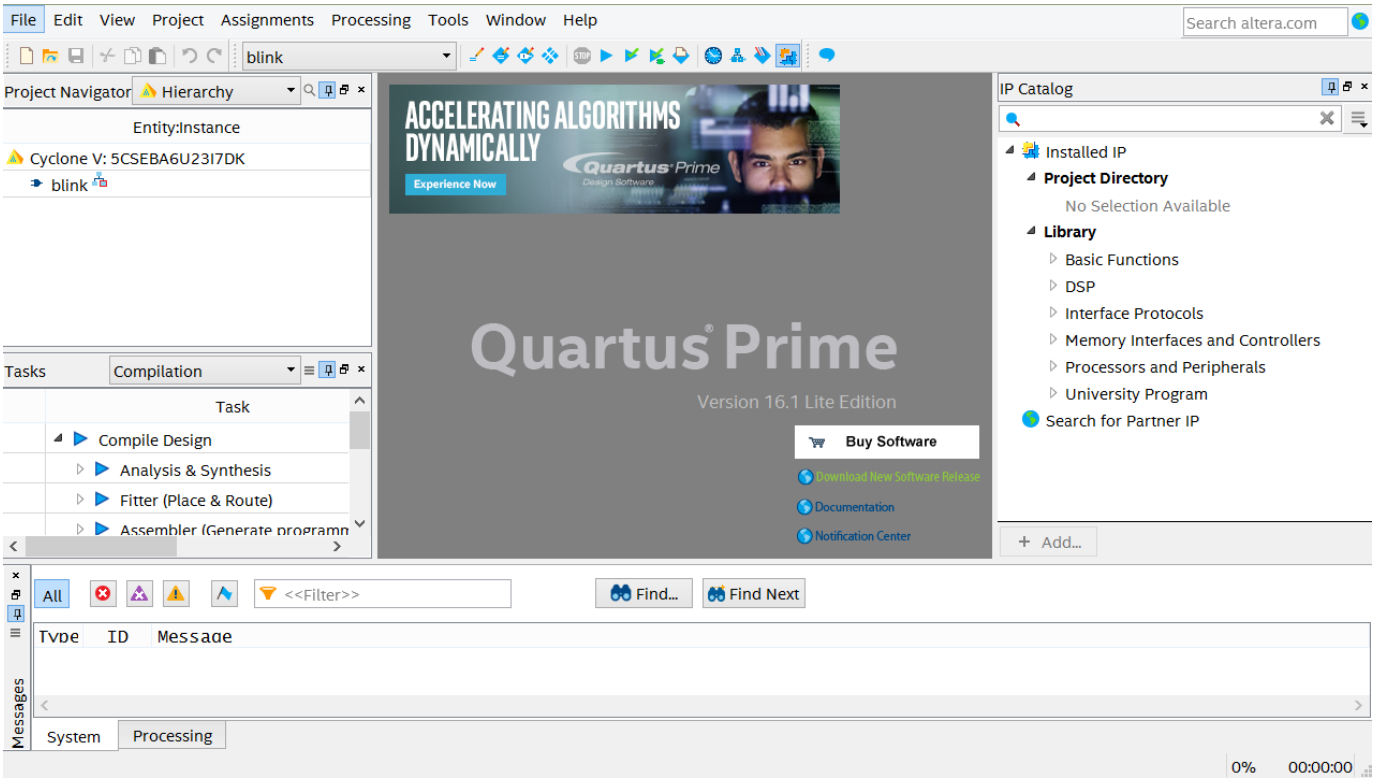
We will be using the default EDA tools and settings so there are no changes to be made.
Click **Next**.




### Step 1.i: Summary

Click **Finish**.

The following screen displays.

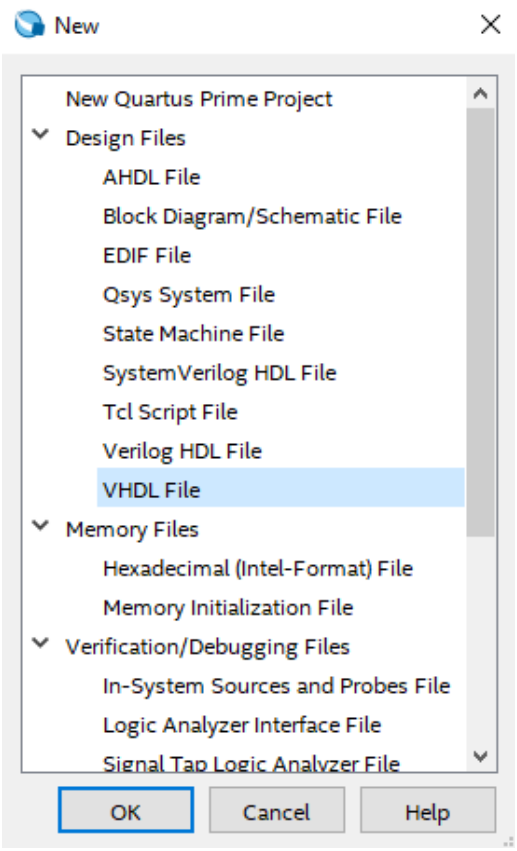


# Step 2: Create an HDL File

## Hardware Description Language (HDL)

We use VHDL as the HDL language but it is also possible to use Verilog or System Verilog.

Step 2.a: Navigate to the File tab (main window), then select **New.**




and

Select **VHDL File,** and then click **OK**.

Step 2.b: Choose **File** > **Save As**. Choose "blink" the file name. **This is your top-level file name and it must match the name of the project name (blink)**. Click **Save**.

for

# Step 3: Create a VHDL Module

Step 3.a: Copy and paste this VHDL code into the blink.vhd window, and then save the file.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.all;

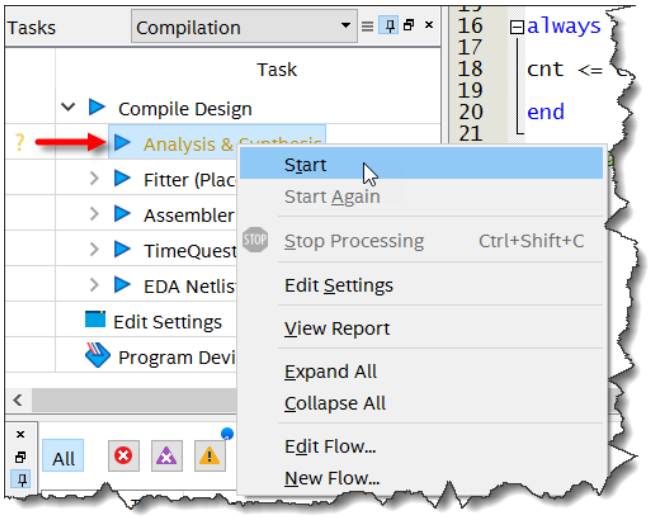
entity blink is
    Port ( clk : in STD_LOGIC;
           rst_n : in STD_LOGIC;
           led : out STD_LOGIC);
end entity;

architecture Behavioral of blink is
signal count : unsigned(24 downto 0);
begin
led <= count(24);
process(clk, rst_n)
begin
 if rst_n = '0' then
    count <= (others => '0');
 elsif rising_edge(clk) then
    count <= count +1;
 end if;
end process;

end architecture;
```

## Step 3.b: Analysis & Synthesis

Right click on Analysis and Synthesis", and then click **Start** to perform a syntax check and synthesis of the VHDL code.

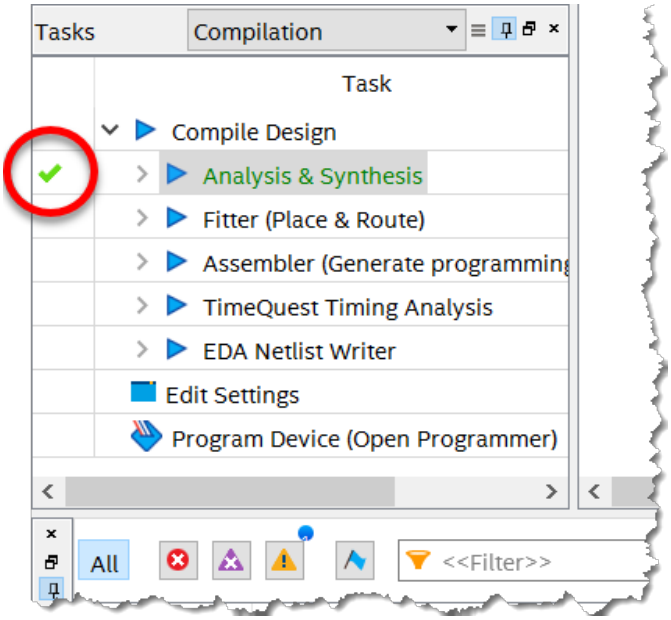



What happens during analysis and synthesis?

The Analysis & Synthesis process:

- Checks design files for syntax and semantic errors
- Performs netlist extraction to build a database which integrates all the design files
- Synthesizes the hardware description language (HDL) code into hardware blocks appropriate for the target FPGA family

If the process completes successfully, a green check mark displays next to Analysis & Synthesis. If you get an error, check your syntax and make sure it matches exactly the code block provided above.



# Step 4: Choose Pin Assignments

Step 4.a: In the top navigation bar, select **Assignments** > **Pin Planner**.

Notice that the input (clk and rst_n) and output (LED) are listed under **Node Name**. This is because you ran the Analysis & Synthesis.

Step 4.b: One at a time, click to highlight the **Location** column for each pin, then type the pin location for the LED, clk and rst_n signals as shown below. The rest of the columns will auto populate with data (some with default values that we'll modify in the next step).

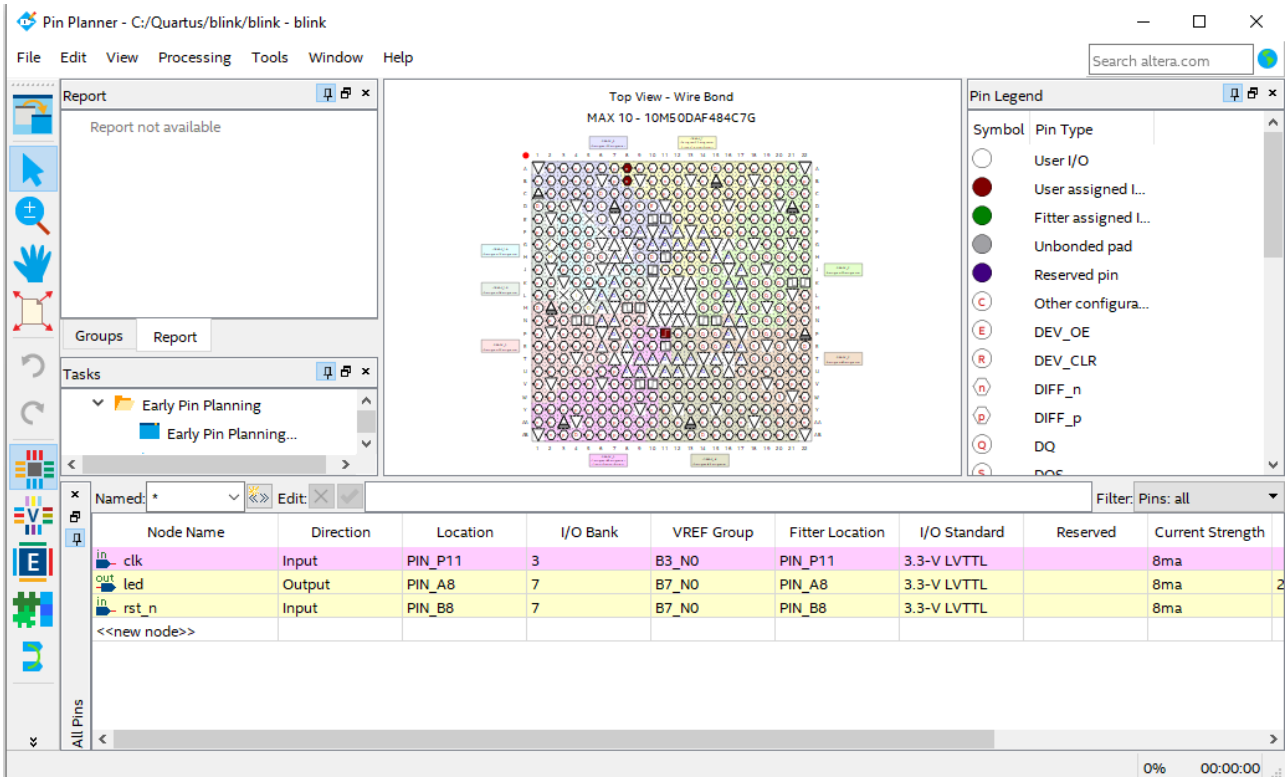
| Node Name | Location |
|---|---|
| LED | PIN_A8 |
| clk | PIN_P11 |
| Rst_n | PIN_B8 |

You also need to change the **I/O standard, Current Strength and Slew Rate** columns from their default settings to the values shown below:

| Node Name | Location | I/O Standard | Current Strength |
|---|---|---|---|
| LED | PIN_A8 | 3.3-V LVTTL | 8mA |
| clk | PIN_V11 | 3.3-V LVTTL | 8mA |
| Rst_n | PIN_B8 | 3.3-V LVTTL | 8mA |

Step 4.c: To change the **I/O standard**, double click each cell and a pull-down menu will appear. Click the down arrow icon and scroll to the desired value. Change the I/O standard from the default 2.5V to **3.3-V LVTTL**.

Step 4.d: The Current Strength will by default read *8ma(default)* and we need to select 8*ma*. If we don't specifically set them, then we get warning messages in our compilation. Set **Current Strength** to **8ma** for both input (clk and rst_n) and output (LED).




Step 4.f: Close the Pin Planner.

# Step 5: Create an SDC File

Before you compile the VHDL code, you need to provide timing constraints for the design. You'll create an SDC (Synopsis Design Constraints) file that contains commands to let the Intel® Quartus® software know how to close timing on the design. Without it, you will get warning messages in the compile flow because the Intel® Quartus® software has no idea how to close timing on the design.

To create a blink.sdc and add it to the blink file directory, do the following.

File ➔ New and choose Synopsis Design Constraints File in the Other Files part. Copy and paste the following content :

```
# inform quartus that the clk port brings a 50MHz clock into our design so
        # that timing closure on our design can be analyzed
create_clock -name clk -period "50MHz" [get_ports clk]
# inform quartus that the LED output port has no critical timing requirements
        # its a single output port driving an LED, there are no timing relationships
        # that are critical for this
set_false_path -from * -to [get_ports LED]
```

Now save the file as  blink.sdc file.

What is an SDC file, and why do I need one?

SDC stands for Synopsys Design Constraints and is an industry standard format which defines the timing constraints for a hardware (silicon) design such as the target frequency of the device, and the timing to external peripherals. The SDC file provides a way for Quartus to verify that the system generated meets its timing requirements.

During synthesis of your FPGA, a design tool called TimeQuest is called by Quartus which reads the timing constraints files, calculates the timing of the internal FPGA signals, and compare these timings to the timing requirements specified by the SDC files. A report is created which verifies timing is met and / or identifies signals which fail to meet timing and require optimization.

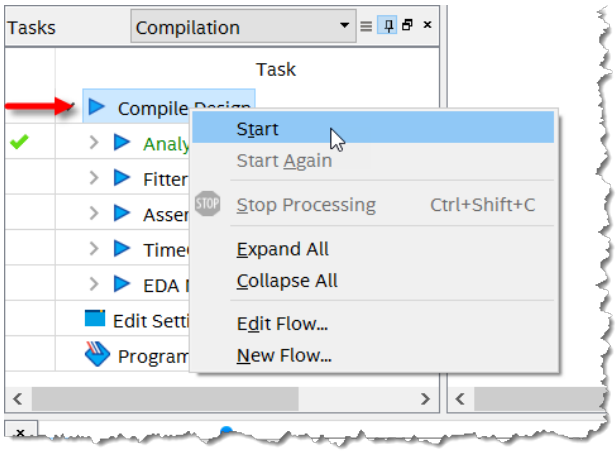
# Step 6: Compile the VHDL Code

## Step 6.a:

Right click on **Compile Design,** and then click **Start**. The tools will then synthesize, place and route, assemble and perform timing analysis on the design Because there are only have a handful of code lines, the compilation should only take a couple of minutes to complete.

What happens during the fitter (place & route)?

The Fitter places and routes the logic of your synthesized design into target device resources. Think of it like a router that lays out a printed circuit board, connecting the various devices together using copper traces. In this case, however, the devices are logic resources (e.g. lookup tables, registers, memories, multipliers, etc.) and the traces are routing "wires" inside the FPGA device.



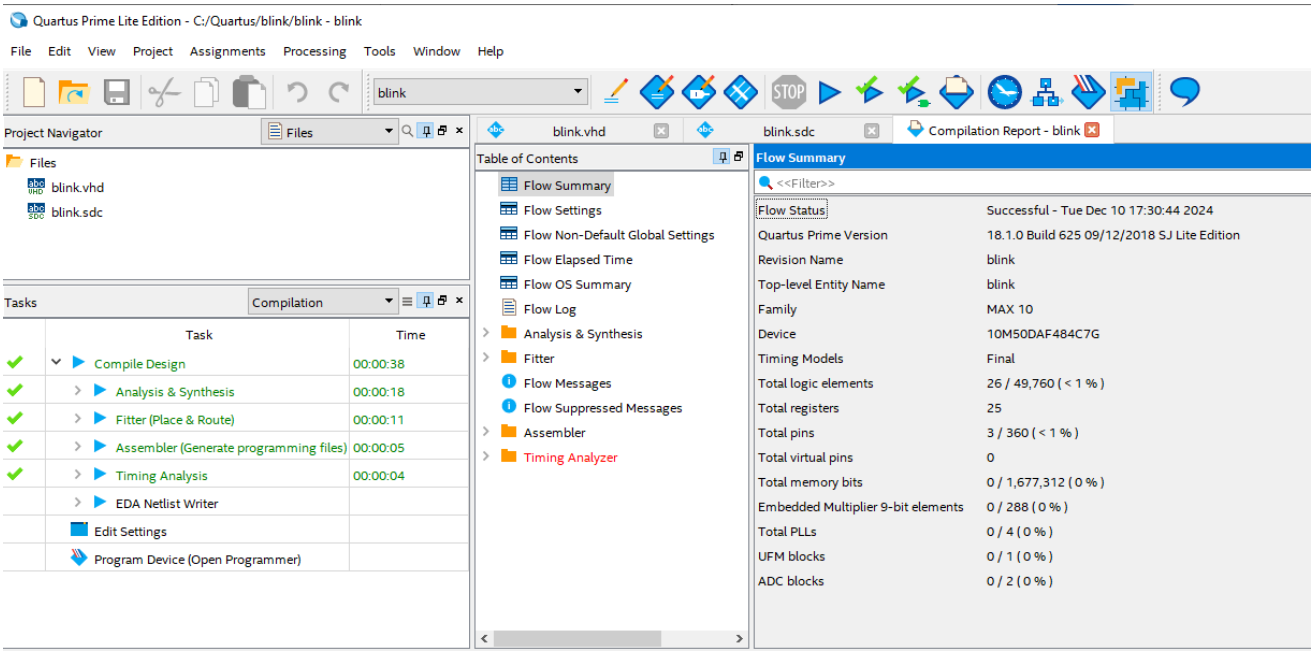

What happens during the assembler?

The Assembler generates a programming image from a successful fit which can then be downloaded to the FPGA device.

What is timing analysis?

Timing analysis is the process of evaluating the timing of logic in the device after it has been synthesized, placed and routed to ensure the all timing requirements are met. The goal is to achieve "timing closure" where all the timing constraints are met.

After compilation is complete you will see a summary report like the one below.

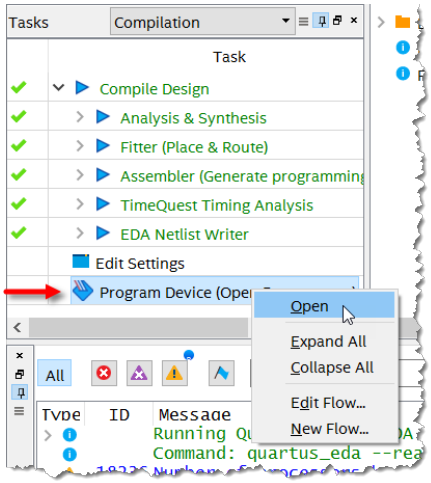


# Step 7: Program the FPGA

The final step is to program the FPGA.

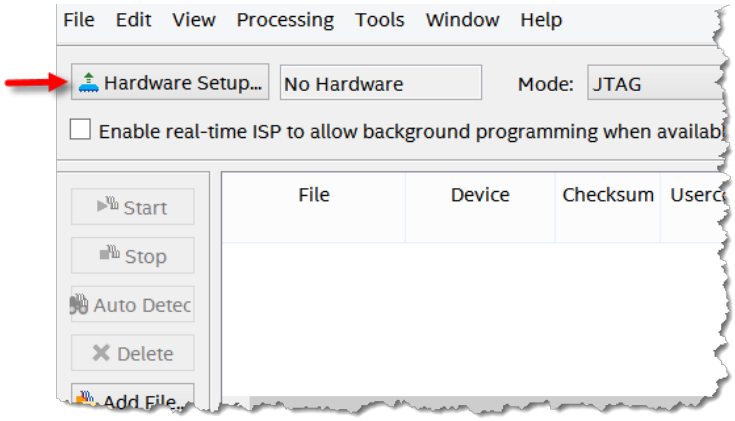
Step 7.a: Connect the board to your computer via the USB blaster port. Use the USB cable that came with the Terasic DE10-Lite kit. Insert the connector into the USB Blaster port on the Terasic DE10-Lite board and the Type-A end into a standard USB port on your host computer.

Step 7.b: Right click to open **Program Device**.

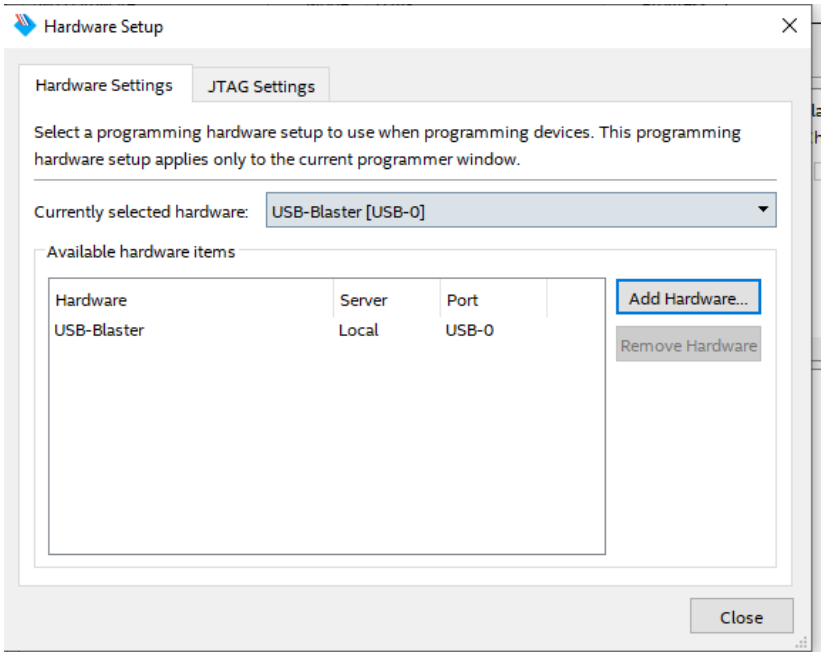



Step 7.c: Hardware Setup

Select **Hardware Setup**.




Under the drop down for **Currently selected hardware,** choose **USB-Blaster**, then click **Close**.



**If you don't see the USB-Blaster in the Hardware Setup, it's perhaps you didn't install the USB Blaster drivers.**

To arrange this, you have to open the **Device manager (Gestionnaire de périphérique)** and update the USB-Blaster driver you should be at the beginning of the list with a yellow triangle that signals a problem.

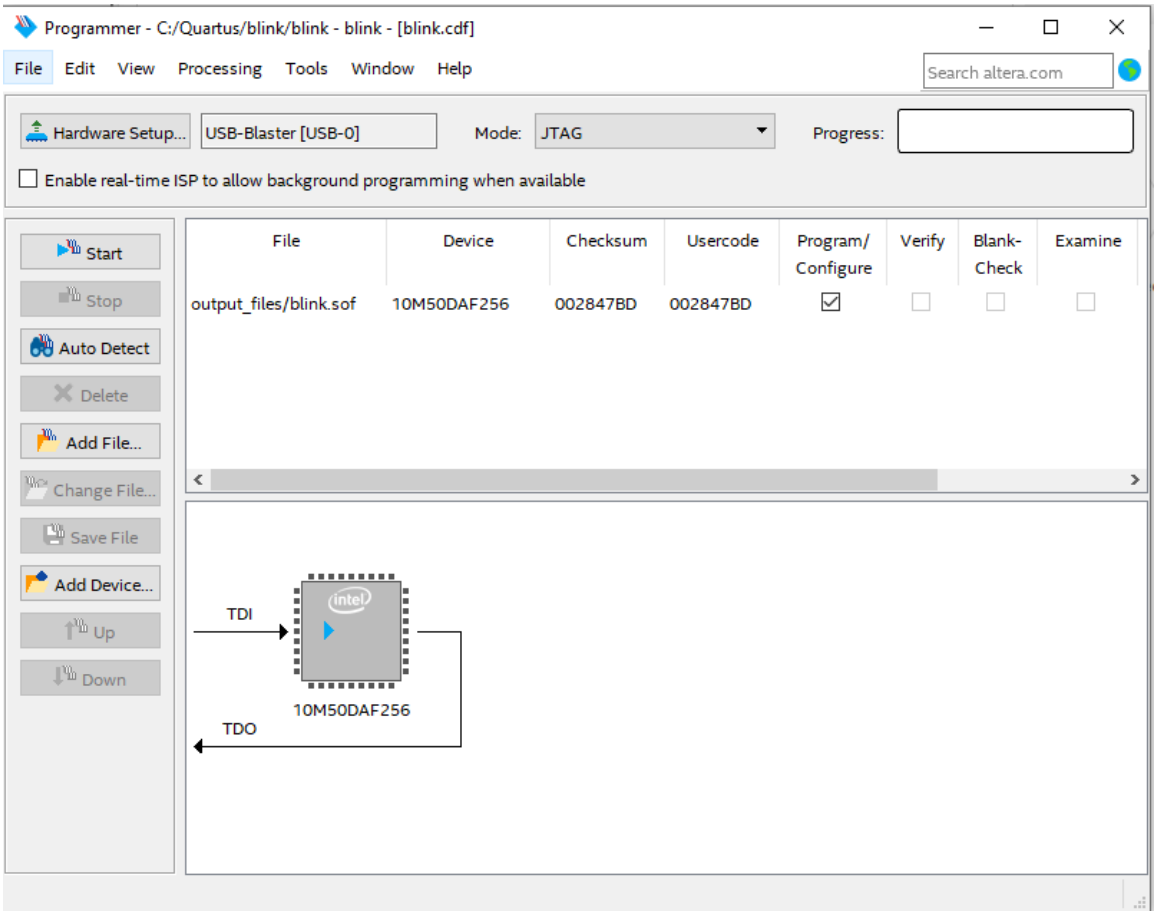
For that, Right click on the **Altera USB-Blaster** and choose **Update the driver.** Then Choose "Browse my computer for driver software" and select the following directory : **C:\intelFPGA_lite\18.1\quartus\drivers**

Click **Next** and it should install the driver for the USB Blaster.

Step 7.f: Add the .sof file.

If the **output/files/blink.sof** file is not already present. Select the **File** column for the **10M50DAF256** device and select **Change File.**

Navigate to the *output_files* folder, select **blink.sof,** and then click **Open**.




Here, **blink.sof** is your programming file. SRAM Object Files (.sof files) are binary files containing data for configuring SRAM-based devices—our FPGA is based on SRAM—using the Intel® Quartus® software Program Device (also called Programmer). The programmer looks into the SOF file and gets the programming bit stream for the device.

Step 7.h: Check the **Program/Configure** column, and then click **Start**.

# Step 8: Observe the blinking LED

If your **progress** bar is 100% (Successful), watch LED0 to go through a blinking sequence.

# Experiment on Your Own

Change the Blink Rate

Now that you've got one successful blinking LED under your belt, you can modify the blink rate of the LED by using a different "bit" of the counter. 1 Hertz is 1/second (cycles per second) and blinking at 1 Hz means the LED blinks once per second. For 2 Hz, the LED blinks twice per second. And 0.25 Hz would blink the LED once every 4 seconds (slow blink). For a slower blink, use a higher bit of the counter and for faster, use a smaller bit (e.g., cnt[22]). Test out different counter bits and see what you get.

Clock and Counter Math

```
cnt[n] where n = the counter bit

2^n; we've chosen n = 24 in our VHDL code sample
2^24 = 16,777,216
```

Our clock is 50 MHZ or 50,000,000

```
Clock / 2^n = blinks per second
50,000,000 / 16,777,216 = 2.9802
```

That's about 3 blinks per second.

To make this change, you will need to:

1. Modify the VHDL file (blink.vhd) and select a different counter bit in the assignment statement.
2. Recompile the design
3. Reprogram the FPGA

### Add More LEDs

Try connecting one or more of the remaining 7 LEDs to other counter bits, with each blinking at a different rate. For example, you could add a new LED connected to counter bit 23 which would blink twice as fast as bit 24. Or you could add all the remaining 7 LEDs and connect each to a unique counter bit.

To make this change, you will need to:

1. Modify the VHDL code (step 3a):
2. Add the new LEDs to the module definition
3. List the new LEDs as being outputs
4. Assign each of the new LEDs to a unique counter bit (cnt[n])
5. Run Analysis and Synthesis (step 3b)
6. Assign the LEDs outputs to pins using the Pin Planner (steps 4b - 4e).

Be sure to set the proper I/O standard and drive strength. The I/O pin connected to the LEDs are as follows:

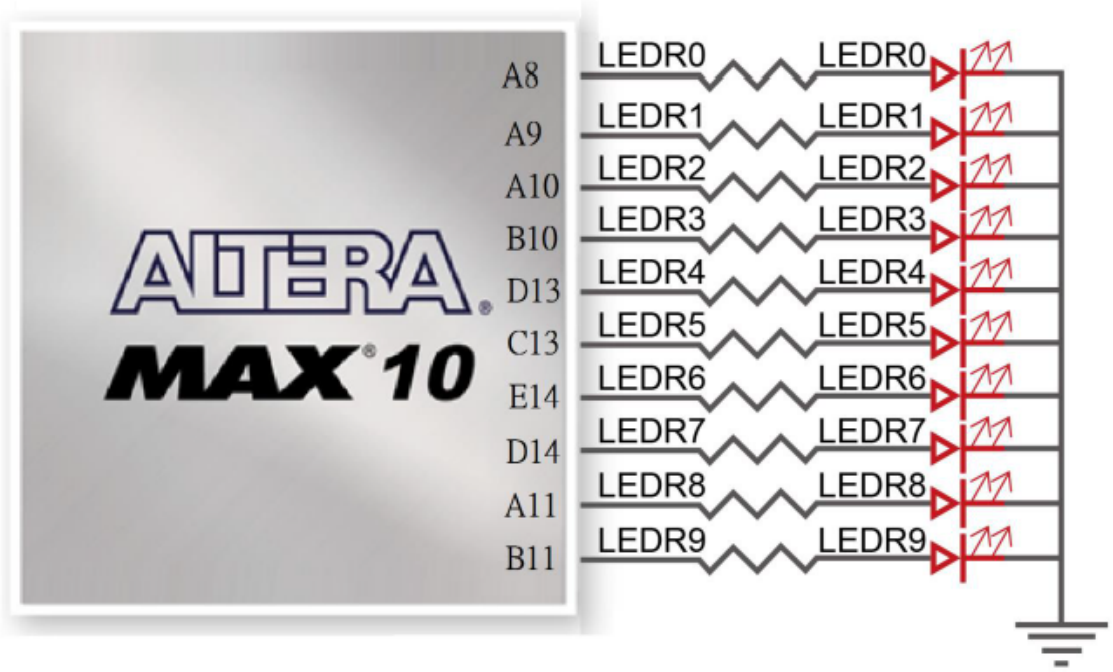



**Figure 3-16 Connections between the LEDs and MAX 10 FPGA**

Table 3-5 Pin Assignment of LEDs

| Signal Name | FPGA Pin No. | Description | I/O Standard |
|---|---|---|---|
| LEDR0 | PIN_A8 | LED [0] | 3.3-V LVTTL |
| LEDR1 | PIN_A9 | LED [1] | 3.3-V LVTTL |
| LEDR2 | PIN_A10 | LED [2] | 3.3-V LVTTL |
| LEDR3 | PIN_B10 | LED [3] | 3.3-V LVTTL |
| LEDR4 | PIN_D13 | LED [4] | 3.3-V LVTTL |
| LEDR5 | PIN_C13 | LED [5] | 3.3-V LVTTL |
| LEDR6 | PIN_E14 | LED [6] | 3.3-V LVTTL |
| LEDR7 | PIN_D14 | LED [7] | 3.3-V LVTTL |
| LEDR8 | PIN_A11 | LED [8] | 3.3-V LVTTL |
| LEDR9 | PIN_B11 | LED [9] | 3.3-V LVTTL |

1. Recompile the design (step 6a)
2. Reprogram the device (step 7h)